School of Computer Science and Artificial Intelligence

Lab Assignment # 09

Course Title : **AI Assistant Coding**
Name of Student : **R Govardhan Sai Ganesh**
Enrollment No. : **2303A54044**
Batch No. : **48**

Lab 9: Documentation Generation – Automatic Documentation and Code Comments

Task 1: Basic Docstring Generation Scenario

You are developing a utility function that processes numerical lists and must be properly documented for future maintenance.

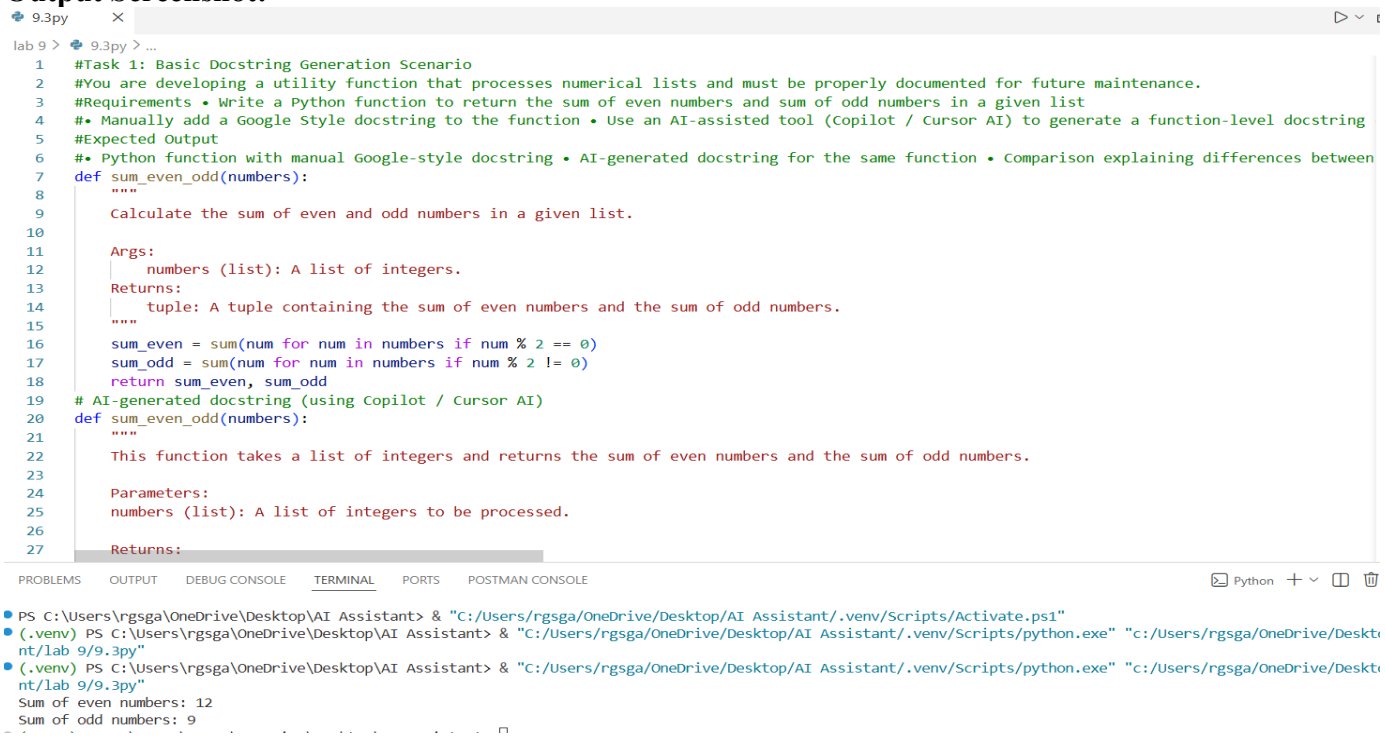
Requirements

- Write a Python function to return the sum of even numbers and sum of odd numbers in a given list
- Manually add a Google Style docstring to the function
- Use an AI-assisted tool (Copilot / Cursor AI) to generate a function-level docstring
- Compare the AI-generated docstring with the manually written docstring
- Analyze clarity, correctness, and completeness

Expected Output

- Python function with manual Google-style docstring
- AI-generated docstring for the same function
- Comparison explaining differences between manual and AI-generated documentation
- Improved understanding of AI-generated function-level documentation

Output Screenshot:



#Comparison of Manual and AI-generated Docstrings:

1. Clarity: Both docstrings are clear and concise, providing a straightforward explanation of the function's purpose and its parameters. The AI-generated docstring is slightly more verbose, which may enhance clarity for some readers.

2. Correctness: Both docstrings accurately describe the function's behavior and the expected input and output. There are no factual errors in either docstring.

3. Completeness: Both docstrings are complete in terms of describing the function's parameters and return values. The AI-generated docstring includes a more detailed description of the return value, which may be considered more complete.

Overall, both docstrings effectively communicate the necessary information about the function. The AI-generated docstring may provide a bit more detail, but the manual docstring is sufficiently clear and correct for most purposes.

Task 2: Automatic Inline Comments

Scenario You are developing a student management module that must be easy to understand for new developers.

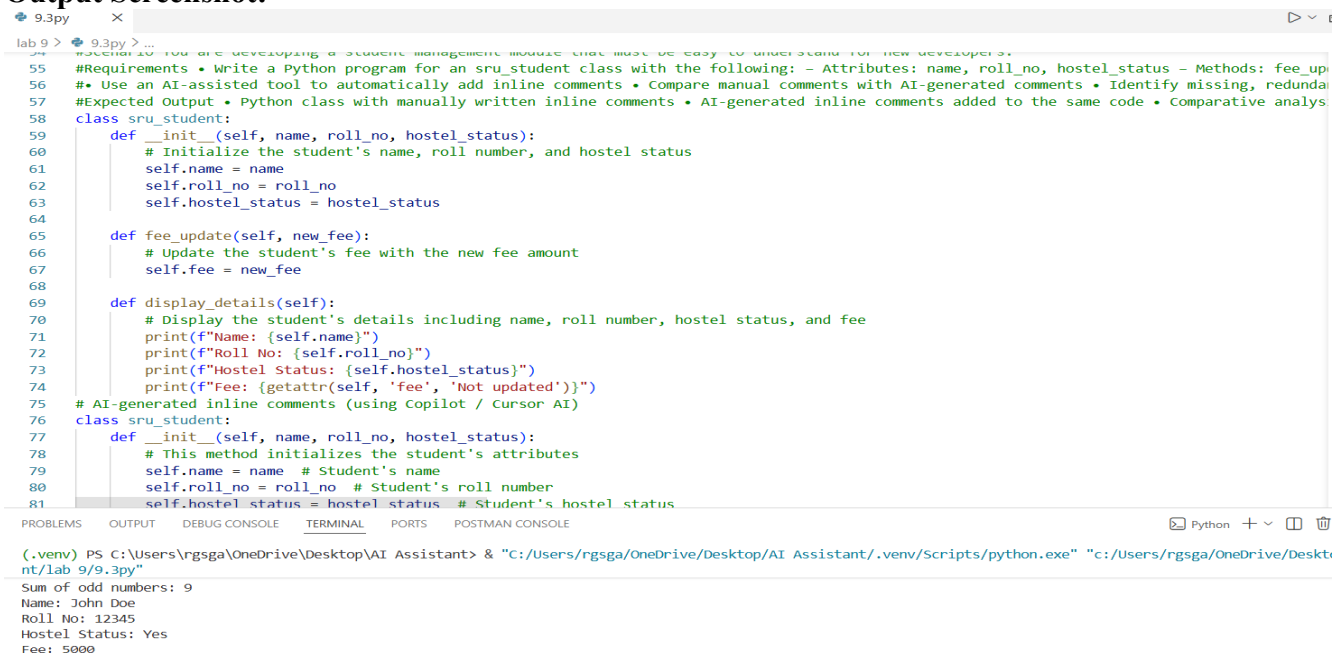
Requirements

- Write a Python program for an sru_student class with the following:
  – Attributes: name, roll_no, hostel_status
  – Methods: fee_update() and display_details()
- Manually write inline comments for each line or logical block
- Use an AI-assisted tool to automatically add inline comments
- Compare manual comments with AI-generated comments
- Identify missing, redundant, or incorrect AI comments

Expected Output

- Python class with manually written inline comments
- AI-generated inline comments added to the same code
- Comparative analysis of manual vs AI comments
- Critical discussion on strengths and limitations of AI-generated comments

Output Screenshot:



 # Comparison of Manual and AI-generated Inline Comments:
1. Clarity: Both sets of comments are clear and provide a good explanation of the code's functionality. The AI-generated comments are more detailed, which may enhance understanding for some readers.

2. Redundancy: The AI-generated comments are more verbose, which can be seen as redundant in some cases. For example, the comment "This method initializes the student's attributes" is somewhat redundant when the code itself is straightforward.

3. Missing Comments: Both sets of comments cover the essential parts of the code, and there are no significant missing comments in either version.

Overall, the AI-generated comments provide a more detailed explanation of the code, which can be beneficial for new developers. However, the manual comments are concise and sufficient for understanding the code's functionality without being overly verbose. The AI-generated comments may be more helpful for those who are new to programming, while the manual comments may be preferred by more experienced developers who value brevity.

Task 3: Module-Level and Function-Level Documentation
Scenario You are building a small calculator module that will be shared across multiple projects and requires structured documentation.

Requirements
- **Write a Python script containing 3–4 functions (e.g., add, subtract, multiply, divide)**
- **Manually write NumPy Style docstrings for each function**
- **Use AI assistance to generate:**
  - **A module-level docstring**
  - **Individual function-level docstrings**
- **Compare AI-generated docstrings with manually written ones**
- **Evaluate documentation structure, accuracy, and readability**

Expected Output
- **Python script with manual NumPy-style docstrings**
- **AI-generated module-level and function-level documentation**
- **Comparison between AI-generated and manual documentation**
- **Clear understanding of structured documentation for multi-function scripts**

 # Comparison of AI-generated and Manual Docstrings:
1. Structure: The AI-generated module-level docstring provides a clear overview of the module and its functions, while the manual function-level docstrings follow the NumPy style, which is well-structured and easy to read.

2. Accuracy: Both the AI-generated and manual docstrings accurately describe the functionality of the module and its functions. There are no factual errors in either version.

3. Readability: The AI-generated module-level docstring is concise and provides a good summary of the module's purpose. The manual function-level docstrings are detailed and follow a consistent format, which enhances readability.

Overall, both the AI-generated and manual docstrings effectively communicate the necessary information about the module and its functions. The AI-generated module-level docstring provides a good overview, while the manual function-level docstrings offer detailed explanations of each

function's parameters and return values. The combination of both types of documentation can provide a comprehensive understanding of the module for users.

Output Screenshot: